

Introduction to Concurrency

- **Concurrency:** Multiple execution flows make progress at the same time by interleaving their executions or by executing instructions at exactly the same time
- **Parallelism:** Multiple tasks running simultaneously, at the exact same time
- Hardware threads dictate the amount of parallelism through parallel execution of multiple kernel threads
- Steps for Parallelisation
 - Decomposition of computation
 - Assigning tasks to threads
 - Orchestration: Communication, synchronisation, scheduling
 - Mapping of threads to physical cores
- **Race Condition:** Timing or order of events affects outcome of executing concurrent code, not necessarily a flaw
- **Data Race**
 - Multiple threads access a shared resource without any synchronisation AND at least one thread modifies it, causing UB
 - Does not occur if both evaluations are on the same thread, both are atomic, or one of them happens before another
- Use mutual exclusion to synchronise access to shared resources
- Other Mechanisms
 - Locks: Primitive with minimal semantics, used as a building block
 - Semaphores: Basic, easy to get the hang of but hard to program with
 - Monitors: High level, requiring language support, implicit operations
 - Messages: Simple model of communication based on atomic transfer of data across a channel
- **Deadlock:** Execution where no progress is made when it should be possible
- Can either ignore it, prevent it, avoid it, or detect and recover from it
- **Starvation:** Thread is prevented from making progress because other threads have resource it requires and initial thread cannot acquire it, usually side effect of scheduling algorithm

Threads and Synchronisation in Modern C++

- Code to Execution:
 - Compilation and linking: Done by compiler
 - Loading: Loader is usually specific to OS
 - Execution: Coordinated by OS, program gets access to resources
- C++ Building:
 - Preprocessor replaces directives
 - Compiler parses C++ code and converts it to assembly
 - Assembler assembles code into machine code
 - Linker produces final compilation output from object files
- **Lifetimes**
 - Begin when storage with proper alignment and size for its type is obtained, and initialisation is complete
 - Ends when object is destroyed, or when destructor call starts, or the storage which the object occupies is released
 - Lifetime of a reference begins when initialisation is complete and ends as if it were a scalar object
- **Ownership**
 - Owner is an object containing a pointer to an object allocated by new for which a delete is required
 - Objects on the free store must have exactly one owner
 - Thread instances own a resource, and instances of threads are movable but not copyable

- **Structure of Program Data**
 - Every variable is an object
 - Every object occupies at least one memory location
 - Fundamental types occupy exactly one memory location
 - Adjacent bit fields are different memory locations but can be part of the same storage unit
- **Invariants:** Statements always true about a particular data structure (might be broken during update)
- Wrap data structure with protection mechanism to prevent data races
- Mutexes provide mutual exclusion and serialisation where threads take turns to access the protected data
- Recommended usage: `std::lock_guard` with `RAII` to unlock on destruction
- Don't pass pointers / references to protected data outside the scope of the lock
- Avoid nested locks and calling user-supplied code while holding a lock; acquire locks in a fixed order
- **Condition Variable**
 - Associated with event or other condition; one or more threads can wait for condition to be satisfied
 - When condition is satisfied, thread notifies one or more threads waiting on condition variable to wake up
 - Condition check should not have side effects
 - **Spurious Wake:** Waiting thread reacquires mutex and checks condition even though it is not notified

Atomics and Memory Model in C++

- Correctly synchronised program should behave as if memory operations are executed in an order that is equivalent to some sequentially consistent interleaved execution of memory operations of each thread, where each write appears to be atomic and globally visible to all processors
- Processor might reorder operations to improve efficiency
- Memory models provide a contract to programmers about what are the visible side effects of their memory operations
- **As-if Rule**
 - The compiler is permitted to perform changes to the program as long as
 - Accesses to volatile objects occur strictly according to the semantics of the expressions in which they occur
 - They are no reordered wrt other volatile access on the same thread
 - At program termination, data written to files is exactly as if the program was simply executed
 - Prompting text which is sent to interactive devices will be shown before the program waits for input
 - Programs with UB are free from this rule

Atomics

- `std::atomic` allows for low-level synchronisation, with operations that reduce to 1 or 2 CPU instructions
- Atomic operations are indivisible and always observed fully done/undone
- Atomic loads will retrieve either initial value of object or value stored by some modification
- A non-atomic operation might be seen as half-done by another thread
- Atomic types provide `is_lock_free()` which checks if operations on a given type are done with atomic instructions, which might be emulated with internal mutexes

Modification Order

- Consists of all writes to an object from all threads in the program, can vary
- With atomic variables, compiler ensures threads agree on MO for each variable, but might still have races
- Once a thread has seen an entry, subsequent reads and writes must return that entry or a later one in the MO

- Threads do not have to agree on separate objects

Memory Order

- C++ Memory Order: Specifies how memory accesses are to be ordered around an atomic operation

Sequentially Consistent Ordering

- Behaviour of program is consistent with a simple sequential view of the world
- All threads must agree on the same order of operations
- Operations within same thread can't be reordered
- Sequentially consistent store synchronises with sequentially consistent load of same variable that reads the value stored
- Performance penalty on weakly-ordered machine with many cores due to need for synchronisation

Building Memory Model

- **Sequenced Before (sb):**
 - Within same thread, A is sequenced before B and will evaluate first
- **Synchronises With (sw):**
 - Appears between atomic load and store operations (`acq + rel`, `seq_cst`)
 - Suitably tagged write / store / release in thread A on a variable synchronises with suitably tagged read / load / acquire in another thread if acquire operations reads value of that write, depends on exact execution
 - Can synchronise with `seq_cst` as well
- **Simply Happens Before (hb):**
 - A sb B
 - A sw B
 - A hb X, X hb B
- **Strongly Happens Before:**
 - Specifies which operations see the effects of other operations
 - If A shb B, then A appears to be evaluated before B in all contexts
 - A sb B
 - A sw B, A, B are `seq_cst`
 - A sb X, X hb Y, Y sb B
 - A shb X, X shb B
- **Visible Side Effects:**
 - A side effect of A on a write M is visible to a read B if
 - A hb B and
 - There is no other side effect X such that A hb X, X hb B
 - The visible sequence of side effects is the longest contiguous subset of side effects to a scalar where B does not hb it
- All modifications to an atomic variable occur in a total order which guarantees coherence
 - WW: If evaluation A modifying atomic M hb B that modifies M, then A appears earlier than B in the MO for M
 - RR: If computation A of atomic M hb a computation B on M, and if the value of A comes from write X on M, then the value of B is either X or the value stored by a side effect Y on M that appears after X in the MO for M
 - RW: If computation A of atomic M hb operation B on M, then the value of A from a side effect X that appears earlier than B in the MO for M
 - WR: If a side effect X on atomic M hb a computation B of M, then the evaluation B takes its value from X or from side effect Y that follows X in the MO for M
- With non-`seq_cst` orderings, there is no single global order of events, and threads only have to agree on MO of each variable

Relaxed Ordering

- Operations with relaxed ordering do not participate in sw relationships
- Operations on same variable within a single thread still obey hb
- Accesses to a atomic variable from the same thread cannot be reordered

Release-Acquire Ordering

- Use acquire and release memory ordering
- Atomic read-modify-write operations use either or both
- No total MO but introduces some synchronisation
- If a store in A with release and a load in B from the same variable with acquire and load in B reads a value written by a store in A, then the store in A sw the load in B
- All memory writes that hb the store in A are visible side effects in B; B must return value that A stored or a later value in the release sequence
- Other threads can still see different memory orders
- **Release Sequence:** The longest continuous subsequence of MO of M that consists of atomic RMW operations made to M after a release operation performed on it (can be relaxed!)
- Sequential Consistency for data race free programs is guaranteed if every atomic uses seq cst

Concurrent Data Structures

- Goal: Multiple threads can access DS concurrently, and each thread sees a self-consistent view
- Thread-Safe DS should ensure no data is lost or corrupted, all invariants are upheld, and no problematic race conditions
- Mutexes are an easy way to provide serialisation but prevent true concurrent access
- Invariants are statements that are always true about a data structure, which might be broken during an update
- Constructors and destructors require exclusive access to DS; user has to ensure this and check whether DS can support assignment, swap, or copy construction
- Minimising the protected region results in fewer operations being serialised and greater potential for concurrency
- Locked-Based Concurrent DS: Ensure right mutex is locked when accessing and lock is held for minimum duration
- **Shared Pointer:** Holds a reference to pointed to data, keeps track of number of references (thread safe), automatically calling destructor once no longer referenced
- Fine-Grained Locking: Analyse constituent parts, associating one mutex with each distinct item, cannot continue using STL library
- To avoid deadlock, avoid nested locks, calling user supplied code while holding a lock, and always acquire locks in a fixed order
- Hand-Over-Hand Locking: Hold lock on current node, acquire next, release current, and continue; can cause deadlock on doubly-linked list

Lock-Free Concurrent Data Structures

- Blocking Data Structures: Use mutexes, condition variables, futures for synchronisation
- Calls library functions which suspend execution until another thread performs an action
- Non-Blocking Data Structures: Do not use blocking library functions
- **Obstruction Free:** If all other threads are paused, any given thread will complete its operation in a bounded number of steps
- **Lock Free:** If multiple threads are operating on a data structure, then after a bounded number of steps, one of them will complete its operation (no deadlock)
- **Wait Free:** Every thread operating on a data structure will complete its operation in a bounded number of steps even if other threads are also operating (no starvation)
- Lock freedom allows individual threads to starve but guarantees system-wide throughput
- An algorithm is lock free if when the program threads are run for a sufficiently long time, at least one of them makes progress

- All wait-free algorithms are lock-free
- Notion of Progress: Function returns, or effects become visible
- More than one thread must be able to access the data structure concurrently
- Wait-Free data structures: Does not result in thread being subject to starvation due to loops, does not involve unbounded number of retries
- Might end up writing spin lock in essence
- Benefits: Maximum concurrency, more robust
- Cons: Might cause livelock, and decrease overall performance due to atomic operations
- Contention and Cache Ping Pong: Changes to data need time to propagate to cache on other core, which might cause a stop and wait depending on memory ordering
- False sharing can produce cache ping pong as well when accessing different memory locations on the same cache line within multiple threads
- Guidelines
 - Prototype using sequentially consistent memory order
 - Use lock-free memory reclamation scheme
 - Watch out for ABA problem, prevalent with free lists (memory pools where nodes are recycled), solve using ABA counter
 - Identify busy-wait loop and help the other thread
- **ABA Problem**
 - Compare and swap checks equality before swapping
 - Suppose our data structure is modified before the CAS check, in a way such that the front happens to have the same address
 - The check passes, although the next pointer on the swapped in node points to garbage
 - To solve this, we can tag on generation counters so that even if the address is the same, the value is different
- **Use After Free**
 - If a thread gets preempted and the head gets deleted while it is trying to access, we might get a UAF
 - Can choose to cause memory leak by not freeing anything
 - Mark nodes for deletion while threads are trying to pop and free all at once
 - Use reference counters and shared pointers
 - Use hazard pointers to track threads and their references
 - We choose to use a recycling center for nodes that should be "deleted", using a lock-free stack
 - This recycling center also needs synchronisation with the queue
- **Serialisation**
 - Due to preempting, it is possible to create a scenario where the actual operations performed do not correspond to a serialised interleaving
 - A pushes but gets interrupted, while B pushes in. B tries to pop, but since A is not finished, the queue still looks empty. Only after this try pop does A finish its work.

Testing and Debugging Multithreaded Programs

- Unwanted Blocking: Deadlock or livelock, or I/O and other external input
- Issues include data races, broken invariants, and lifetime issues
- Techniques for testing include stress testing, or special implementations for synchronisation primitives
- Test scalability and performance as well
- Useful tools for debugging memory issues include valgrind and sanitisers
- **Valgrind**
 - Memcheck uses shadow memory which tracks information on memory usage and is used to detect and report incorrect accesses
 - A-bit (Valid-Address): Corresponding byte accessible
 - V-bit (Valid-Value): Corresponding bit initialised
 - Ensure each allocation is freed only once, each access is to a currently allocated space, and all reads are to locations already written
 - Helgrind detects misuses of the pthread API, potential deadlock from lock

- ordering, and data races by intercepting function calls
- Builds directed graph of lock acquisition
- Checks order in which memory accesses happen, building DAG of happens before relationships which is used to flag cycles
- **Sanitisers**
 - Different types of sanitisers such as ASan (address), LSan (memory leak), TSan (thread), MSan (uninitialised memory), HWASAN, UBSan (undefined behaviour)
 - ASan uses shadow values to represent individual bytes allocation
 - Any aligned 8 bytes can have N good bytes and $8 - N$ bad bytes giving 9 states
 - TSan adds checks at every function entry and exit
 - It performs direct shadow mapping from actual addresses to shadow addresses
 - Each 8 byte shadow cell represents one memory access, storing TID, Epoch, Position in word, IsWrite
 - Uses this information to determines hb and races

Concurrency in Go

- Model programs in terms of tasks and synchronise them by making them communicate
- Many different concurrent designs possible
- Finer level of granularity enables program to scale dynamically when it runs due to amount of parallelism possible
- Task parallelism: Do the same work faster
- Data parallelism: Embarrassingly parallel algorithms, do more work in same amount of time
- Task dependency graph: DAG representing control dependencies between tasks
- Critical Path Length: Maximum / Slowest completion time
- Degree of Concurrency: Total Work / Critical Path Length
- Speedup: $S_p(n) = \frac{T_{best_seq}(n)}{T_p(n)}$
- Amdahl's Law: $S_p(n) = \frac{1}{\frac{f}{f+(1-f)/p}} \leq \frac{1}{f}$, f is sequential fraction
- Go is based on Communicating Sequential Processes
- Concurrency is found in Goroutines: independently running functions on OS threads
- Communication is found in Channels paired with select statements
- Goroutines follow the fork-join model and share the same address space
- Runtime multiplexes goroutines onto OS threads, decoupling concurrency from parallelism
- Goroutines are special coroutines (concurrent subroutines), preemptable
- Goroutines take a few kilobytes and are relatively lightweight but not garbage collected
- Channels are streams of information, are actually references to places in memory where the real channel resides
- Channels are typed, bi/unidirectional, and blocking
- Owner is the goroutine that instantiates, writes, and closes a channel
- Owners have write view into unidirection channels while utilisers only have read view
- Ownership increases safety The select statement allow sus to compose channels to form larger abstractions, similar to a switch
- select blocks if no channels are ready
- The Sync package provides typical synchronisation constructs which are used mostly in small scopes
- Go memory model specifies conditions under which reads of a variable can be guaranteed to observe values produced by writes to the same variable in different goroutines
- Go has DRF-SC
- Within a goroutine, reads and write follow sequenced before

- Read `r` is guaranteed to observe write `w` if `w` happens before `r` and any other write to the variable happens before `w` or after `r`
- **Happens before** is the transitive closure of the union of sequenced before and synchronised before
- **Synchronised before**:
 - The `go` statement starting a new goroutine `sb` the goroutine's execution
 - The exit of goroutine is not guaranteed to be `sb` any event
 - Send on a channel is `sb` completion of corresponding receive
 - Closing of channel is `sb` receive that returns zero value
 - Receive from unbuffered channel is `sb` completion of corresponding send
 - `kth` receive on a channel with capacity `C` is `sb` $(k+C)$ th send from that channel completing

Concurrency Patterns in Go

- **Confinement**:
 - Achieve safe operation via synchronisation
 - Safe concurrency with good performance through immutable or protected data
 - Ad-hoc confinement: By convention, data is only modified from one goroutine
 - Lexical confinement: Restrict access to shared locations
- **For-select loop**: Sending iteration variables out on a channel, looping and waiting to be stopped
- Preventing leaks: Goroutine responsible for creating a goroutine is also responsible for ensuring it can stop it
- Error handling: Gracefully handle erroneous states, errors should be tightly coupled with result types and passed along through the same lines of communication
- **Pipeline**:
 - Set of data processing elements connected in series, where output of one element is the input of the next one
 - Series of stages connected by channels
 - Each stage is a group of goroutines running the same function
 - Separate concerns of each stage
 - Each stage should take roughly equal times to complete their tasks
 - Fan-out to decrease processing times for bottleneck stages
 - Tweaking needed to be more efficient than task pool
 - Pipeline is better if there is a cap on a specific resource needed by all tasks in the task pool
- **Fan-out, fan-in**:
 - Solves problem of uneven pipeline stages
 - Fan-out: Start multiple goroutines to handle input from pipeline
 - Fan-in: Combine multiple results into one channel
 - Fan-out if stages don't rely on intermediate values, and they take a long time to run
 - No guarantee on running or return order

Go Scheduler

- Multiplexing goroutines onto OS threads is done through work stealing
- Fair scheduling: Equally divide tasks to number of processors
- Centralised or Decentralised work queues
- G: goroutines, placed in global run queue (GRQ) when runnable
- M: OS thread, runs goroutines from local run queue (LRQ), context switching among multiple goroutines
- P: logical processor
- Preemptive scheduler schedules goroutines and garbage collection
- Waiting: goroutine stopped and waiting for something
- Runnable: goroutine wants time on M
- Executing: goroutine placed on M and executing
- Networking system calls can be asynchronously processed, network poller handles it, no new Ms are created which reduces scheduling load

- When goroutine G1 blocks on M1, scheduler detaches M1 from P and brings in new M2 to run, G1 can join back LRQ when unblocked
- Work Stealing: Ms that are done with their LRQ can steal from the front of LRQs that belong to other Ms

Classical Synchronisation Problems

- Model common problems faced in computer systems
- **Producer Consumer**: Processes share a buffer; producers insert while consumers remove from it
- **Readers Writers**: Processes share a data structure; reader retrieves information while write modifies information, writers must have exclusive access
- **Barrier**: Any thread must stop at this point and cannot proceed until all other threads reach this barrier
- `std::latch` is a single use barrier while `std::barrier` is reusable (combining tree barrier, fan-in style)
- **Dining Philosophers**: Need both chopsticks to eat, need some sort of deadlock avoidance algorithm e.g. odd-even ring communication
- **Barbershop**: If no customers, barber sleeps, if barber busy, customer waits in free chairs, if not, customer leaves

Safety in Concurrent Programming with Rust

- Goals: Safety in systems programming, painless concurrency
- Provides strong safety guarantees like no segfaults or no data races
- No compromises on performance, no garbage collector or runtime
- C++ is unsafe, references are left dangling when underlying object is freed
- Aliased pointers: pointers that point to the same chunk of memory
- Mutation: Changing a pointer
- Aliasing + Mutation: Modifying pointers that point at the same chunk of memory
- Solution: Ownership and borrowing, preventing simultaneous mutation and aliasing
- Ownership does not allow aliasing
- Giving ownership is the default, something similar to move, but enforced at compilation time
- Deep copy can be done with `clone()`
- Shared borrow allows aliasing, but no mutation, "don't break your friend's toys"
- Mutable borrow allows mutation but no aliasing, "no it's my turn now"
- Borrow checker statically prevents aliasing and mutation at compile time
- Ownership prevents double-free, borrowing prevents use-after-free
- Data race is caused by sharing, mutation, and no ordering; rust prevents both sharing and mutation from happening at the same time
- Rust offers library based concurrency (not language based)
- Rustup installs rust tools, Rustc is the rust compiler, Cargo is the Rust package manager
- Packages consists of one or more crates that provide functionality, crates are binaries or libraries, modules are used to organise code within a crate into groups
- Rust enforces lifetimes at compile time, values live in scopes and references must not outlive values they point to
- static: data does not borrow anything that could go out of scope
- Data moved into a thread must be owned or only contain static references
- Reference counted types are similar to shared pointers in C++
- RC types are not atomically managed and do not have a `Send` trait (interface)
- `Send` trait: Transferred across thread boundaries
- `Sync` trait: Safe to share references between threads, iff `&T` is `Send`
- `Copy` trait: Safe to memcopy, not for Strings
- **Atomic Reference Counted (Arc)** allows only for shared references, keeps reference counter
- Threads require `T: Send + 'static`, ensuring no borrowed stack data, and safe execution beyond current scope

- Mutexes are based on ownership, locking returns a guard; a proxy through which we can access the data
- Atomics are similar to C++
- `mpsc::channel` provides a multi-producer, single-consumer FIFO queue
- **Crossbeam** provides ability to create scoped threads, now in std, and message passing with MPMC channels and exponential backoff
- Scopes are containers to put threads in; cannot borrow variables mutably in two threads in the same scope
- **Rayon** is a data parallelism library, similar to OpenMP

Asynchronous Programming in Rust

- Closures can be called like functions and be passed as parameters and return values
- Closures consist of code and capture environment variables, and are eagerly evaluated
- Context switching is expensive, and thread stack spaces add up
- Non-blocking I/O enables concurrency even if we only have one thread
- `epoll` is a kernel provided mechanism that notifies us of what file descriptors are ready for I/O
- **Futures** allow us to keep track of in-progress operations along with associated state in one package
- Futures represent a value that will exist sometime in the future
- Event loops are runtime for futures which will repeatedly poll it until ready
- Rust provides zero cost abstractions for futures
- Polling a future returns an `enum` which is either ready or pending
- Context is passed into `poll` which includes a `wake()` function, called when future can make progress again, implemented using system calls
- When `wake()` is called, executor can use context to see which future can be polled to make new progress
- Executors loop over futures that can currently make progress, continually calling `poll()` before waiting, then waiting until `wait()` is called
- Futures should not block
- Executor runtimes like **Tokio** provide non-blocking implementations for synchronisation primitives
- Futures can be composed with combinators
- Async functions return futures
- `Await` waits for a future and gets its value
- State management can be done with `enums`
- Async functions have no stack, state is self-contained, no recursion is possible
- Async makes sense when you need a high degree of concurrency and work is primarily I/O bound

Checking Concurrent Programs

- Model checking: Build model using a specific Domain Specific Language (DSL), then check constraints
- Formal methods are rigorous and exhaustive, but tedious and time consuming
- Use a model checker or proof assistant and use specification to write the code
- Add formal specification (invariants) as comments
- Temporal Logic of Actions+ (TLA+ (TLC)) focuses on temporal properties and is good for modeling concurrent systems
- Coq Proof Assistant generates `OCaml`, `Haskell`, `Scheme`, and is good for interactive proof methods
- Alloy (alloy analyser) focuses on relational logic and is good for modeling structures
- TLA+ is a high level language for modeling programs and systems, based on the idea that the best way to describe things precisely is to use simple mathematics
- Specification is written before being exhaustively testing the states, then code is written based on TLA+ spec
- Model checker checks safety using basic set theory and liveness using temporal logic

- T: Temporal (time), L: Logic (Boolean logic), A: Actions (state machines), +: Plus (some stuff)
- Predicates are expressions returning Booleans
- Actions are state machines with states and transitions
- Actions are tests
- Temporal models state transitions over time
- Temporal properties apply to the whole system over time

Distributed Systems

- Concurrent programs are everywhere due to the usage of distributed systems
- System where multiple processes located on networked computers communicate via messages to achieve a common goal
- Challenge: No global clock or ordering of events, use logical clock / memory models
- Challenge: Resilience - Consistency - Consensus, errors due to concurrent nature of system

Transactional Memory

- Declarative: Programmer defines what needs to be done
- Imperative: Programmer defines how it should be done
- Transactional memory relies on memory transactions which are atomic and isolated sequences of memory accesses
- Often supported at hardware level
- Thread safe: Free of data races
- Atomicity: All memory writes in transaction take effect at once
- Isolation: No other thread observes writes before a transaction commits
- Serialisability: Transactions appear to commit in a single serial order, and all threads observe the same order
- Progress: Ensure transactions continue to make forward progress, either completing their work or making measurable steps towards completion
- **Conflict Detection:**
 - Detect and handle conflicts between transactions, track read and write sets
 - Pessimistic Detection: Check for conflicts immediately during loads or stores, contention manager stalls or aborts transactions when conflict detected
 - Undo less work, turn some aborts to stalls, but no forward progress guarantees, needs fine-grained communication
 - Optimistic Detection: Detect conflicts when transaction attempts to commit, give priority co committing transaction
 - Guarantees progress, bulk communication, but detects conflicts late and can still have fairness problems
- **Data Versioning:**
 - Manage uncommitted and new committed versions of data for concurrent transactions
 - Eager Versioning: Use undo log, restore from it on conflict
 - Lazy Versioning: Use write buffer, copy from write buffer at commit
- Transaction Data Structures
 - Transaction Descriptor (per thread): Used for conflict detection and operations, includes read / write set, undo log / write buffer
 - Transaction Record (per data): Pointer sized record guarding shared data, tracks transactional state of data
- Conflict Detection Granularity:
 - Object Granularity: Low overhead mapping operation, exposes optimisation opportunities, false conflicts possible
 - Element / Field Granularity (word): Reduces false conflicts, improves concurrency, increased overhead
 - Cache Line Granularity (multiple words): Matches hardware, reduces storage overhead, hard for programmer and compiler to analyse
 - Mix and Match per type basis

Keith



Gopher



Ferris



Prof Cristina

